

Module II: Process Management

Understanding processes, their lifecycle, control blocks, and context switching in operating systems

Process Concepts

Programs in execution, process states, and lifecycle

PCB & States

Process Control Block structure and state transitions

Context Switching

Saving and restoring process states efficiently

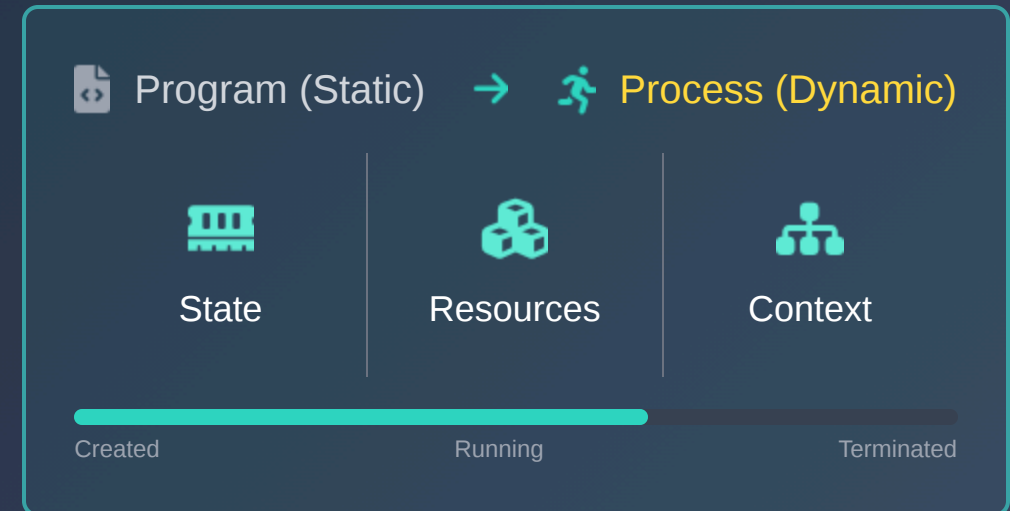
What is a Process?

A **process** is an instance of a program in execution, representing an active entity with its own state, resources, and execution context.

Unlike a static program file, a process is dynamic, evolving through distinct states during its lifecycle.

Key Characteristics:

-  Dynamic entity that changes state during execution
-  Managed by the operating system for multitasking
-  Has unique identity and isolation from other processes
-  Enables resource sharing and efficient CPU utilization



Program vs Process

The key distinction between a **program** and a **process**:

Feature	 Program	 Process
Nature		Passive collection of instructions stored on disk
Behavior		Inert, remains static until executed
Execution		No execution flow, consumes no CPU time
Resource Usage		Inactive, no resources allocated
Multiple Instances		Single instance on disk

 *Example: Multiple browser windows represent separate process instances of the same program.*

Process Attributes and Memory Layout

Process Attributes



Process ID (PID)

Unique identifier assigned by the OS for process tracking



Open Files

List of files currently accessed by the process



Accounting Information

CPU time used, start time, execution limits, resource consumption



Scheduling Priority

Determines the order in which the process is selected for execution

Memory Layout

Text Segment

Executable Code

Contains the compiled instructions of the program

Data Segment

Global/Static Variables

Stores global variables and static data

Heap

Dynamic Memory

Used for dynamically allocated memory during runtime

Stack

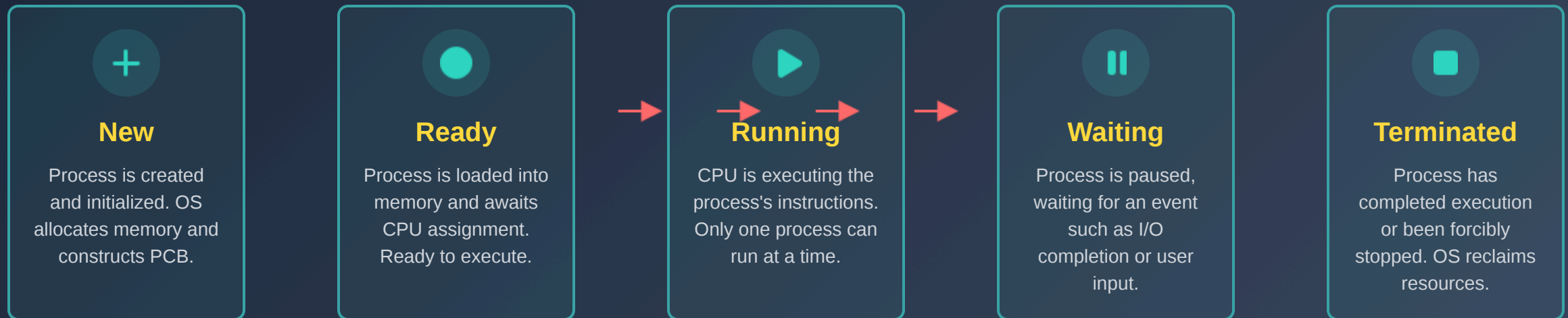
Local Variables

Stores local variables and function call information

 Each process has its own memory layout, ensuring isolation between processes

Process Lifecycle Overview

The lifecycle of a process spans several well-defined stages, managed by the operating system to enable efficient multitasking and resource utilization.



 This lifecycle model provides a framework for process management, enabling efficient CPU scheduling and resource allocation.

Process States: New and Ready

New State

The initial state when a process is created.

- ✓ OS allocates memory and initializes PCB
- ✓ Process is not yet eligible for CPU scheduling
- ✓ Resides in secondary storage until loaded into main memory
- ✓ Transition to Ready state occurs when OS places it in the ready queue



Ready State

Process is loaded into main memory and waiting for CPU assignment.

- ✓ Process is ready to run but must wait for CPU scheduler selection
- ✓ Multiple processes may exist in this state, forming the ready queue
- ✓ Process remains here until the scheduler selects it for execution
- ✓ Transition to Running occurs when CPU is allocated to the process



Process States: Running and Waiting

Running State

The CPU is **actively executing** the process's instructions.

Characteristics:

- ✔ Only one process can be running on a single-processor system
- ✔ Process executes from program counter
- ✔ Performs computations, accesses memory, or makes system calls

CPU Execution



Transitions

 Preemption
Time slice expiration

 Waiting
I/O request

Waiting State

The process is **blocked** and waiting for an external event.

Characteristics:

- ✔ Paused until specific event occurs
- ✔ Cannot proceed until event completion
- ✔ Moved to I/O queue or wait queue

I/O Operation



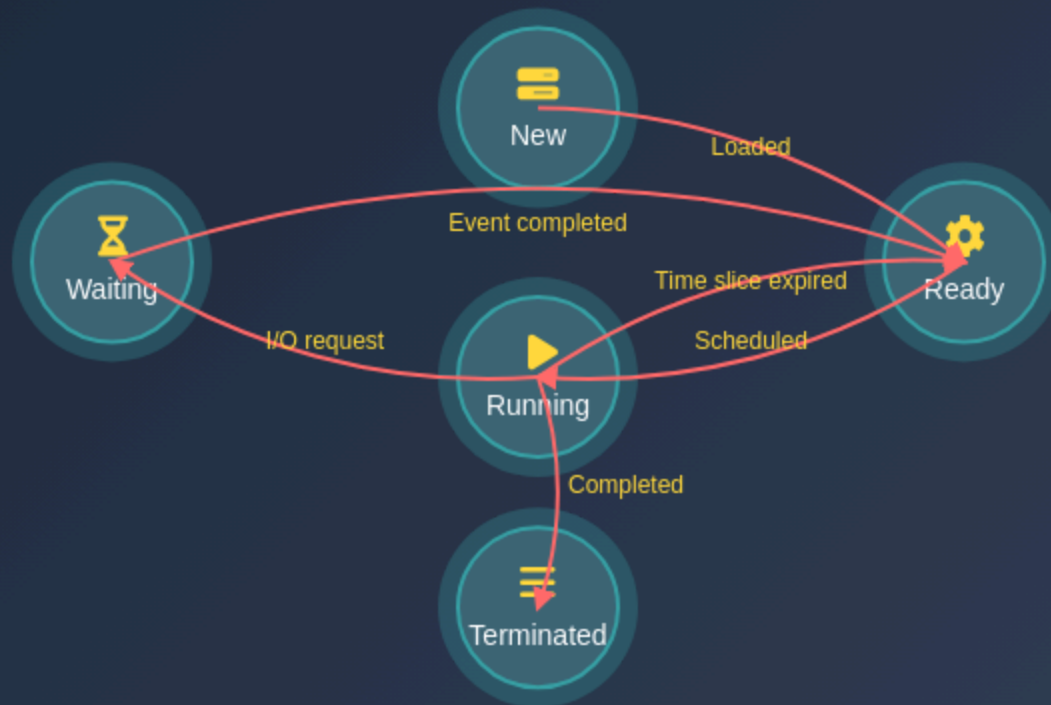
Transitions

✔ Ready
Event completion

 Terminated
Force kill

Process State Transitions

Transitions between states are driven by system events and scheduling decisions, managed by the operating system's schedulers and interrupt handlers.



Key Transitions:

- New → Ready
Process loaded into memory and added to ready queue
- Ready → Running
CPU scheduler selects process for execution
- Running → Ready
Process preempted due to time slice expiration
- Running → Waiting
Process requests I/O or waits for an event
- Waiting → Ready
Event completes, process becomes eligible
- Running → Terminated
Process completes or terminated by OS

Process Control Block (PCB) Structure

The **Process Control Block (PCB)** is the central data structure used by the operating system to manage and track each process. It contains all essential information required for process control, context switching, and system scheduling.

Process State

Current status (New, Ready, Running, Waiting, Terminated)

Process ID (PID)

Unique identifier assigned by the OS

Parent Process ID (PPID)

Identifies the creating process

Program Counter (PC)

Address of next instruction to execute

CPU Registers

Values of all general-purpose registers

Memory Management

Page tables, segment tables, base/limit registers

Scheduling Info

Priority, time quantum, queue pointers

Accounting Information

CPU time, start time, execution limits

I/O Status Information

Open files, allocated devices, pending I/O

 The PCB is stored in protected kernel memory and is accessed during every process transition. While it consumes memory, its benefits in system stability and security far outweigh the cost.

PCB Components and Functions

The Process Control Block (PCB) contains essential information that enables process management, context switching, and system scheduling.



Process Identification

Unique PID and PPID for process tracking and hierarchical process management.

Arpaci-Dusseau '23



CPU State Management

Saves register values and program counter during context switches.

Silberschatz '20



Memory Management

Page tables and segment tables for virtual memory management.

Tanenbaum '22



Accounting Information

Tracks CPU time and execution statistics for scheduling decisions.

Love '21



Scheduling Information

Priority and time quantum for process scheduler decisions.

Arpaci-Dusseau '23



I/O Status Information

Open files and allocated I/O devices for resource management.

Silberschatz '20



Program Counter

Points to the next instruction to execute, saved during context switches.

Love '21



Process State

Current status (New, Ready, Running, Waiting, Terminated).

Tanenbaum '22

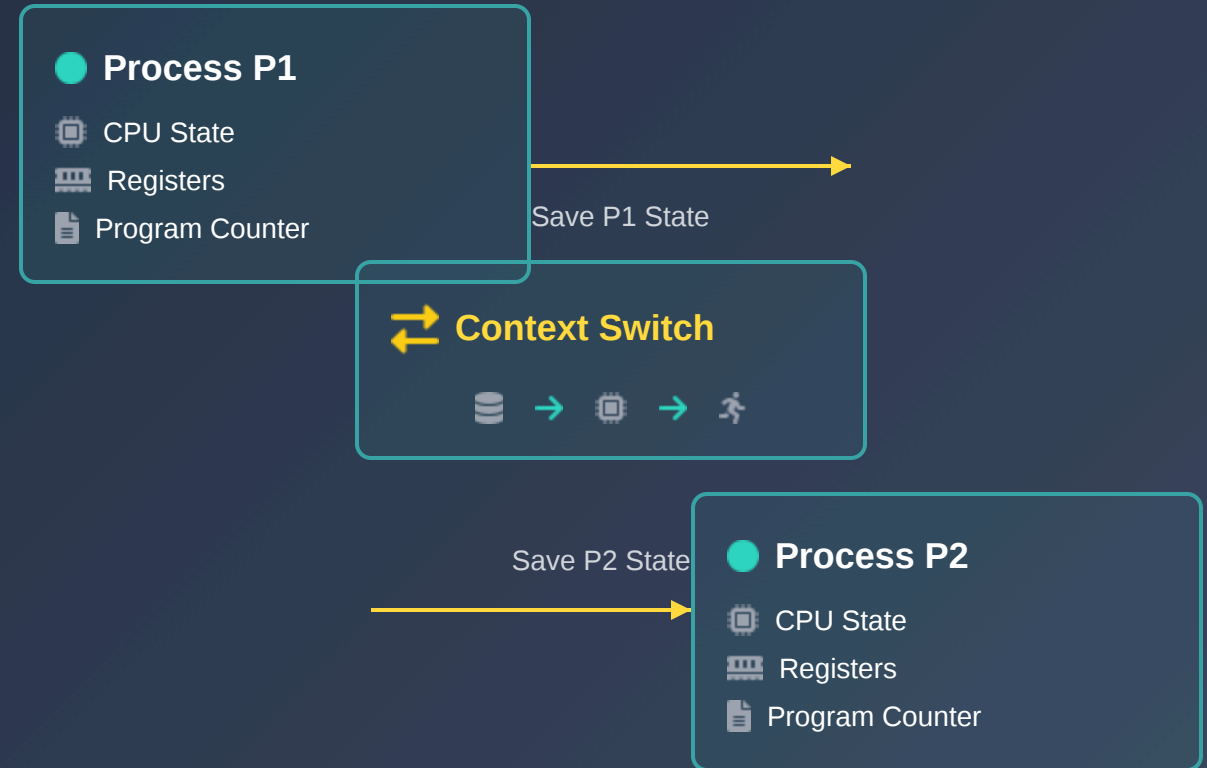
Context Switching Overview

Context switching is the mechanism by which the CPU switches from one process to another, enabling multitasking.

It involves saving the current process state and restoring the next process state.

Key Aspects:

- 💾 Saves CPU registers, program counter, and other state information
- 🔄 Enables efficient multitasking in preemptive systems
- 🛡️ Prevents one process from monopolizing the CPU
- ↔️ Essential for system responsiveness and resource utilization



Context Switch Triggers

Context switches are triggered by several system events, enabling multitasking in operating systems.



Timer Expiration

Time-sharing systems allocate time slices. When timer expires, scheduler preempts the running process.



Interrupts

Hardware or software interrupts cause CPU to pause current process and handle the event.



Process Preemption

Higher-priority process becomes ready, prompting scheduler to switch to it.



I/O Blocking

When a process requests I/O, it enters waiting state, requiring a switch to another process.

Context Switch Process Steps

Context switching involves saving the current process state and restoring the next process state for seamless execution continuation.

1 Save Current State

-  OS saves CPU registers, program counter, stack pointer into current process's PCB
-  Preserves exact execution context for later resumption

2 Update & Select

-  Update process state in PCB (e.g., from Running to Ready)
-  Scheduler selects next process from ready queue based on scheduling algorithm

3 Restore & Resume

-  Retrieve saved state from new process's PCB and load into CPU registers
-  Update new process state to Running and resume execution from saved PC



 **Key Point:** Context switching ensures each process can resume execution exactly where it left off, maintaining correctness and continuity.

Context Switching Overhead

Context switching incurs significant overhead that can degrade system performance if excessive.

Key Components of Overhead:



Register Save/Restore

Saving and restoring CPU registers takes time, especially with many registers



Memory Access

Accessing the PCB in memory adds latency to the switching process



TLB Flush

When processes use different virtual address spaces, TLB must be flushed, causing cache misses



Scheduler Overhead

The scheduler must evaluate the ready queue and make decisions

Context Switching Process



Save State



Schedule



Restore State



Thrashing Issues

Excessive context switching can lead to **thrashing**, where the system spends more time switching between processes than executing them.



Poor Responsiveness



Reduced Throughput



High CPU Usage

Hardware Support and Optimization

Hardware Support



Multiple Register Sets

Some architectures maintain multiple CPU register sets. A switch only requires changing the active set pointer.



Task State Segment (TSS)

Intel x86 processors support TSS for hardware task switching, though most OSes avoid it due to inefficiency.



Fast Register Access

Optimized instruction sets allow rapid saving and loading of register states during context switches.

Optimization Techniques



Use efficient scheduling algorithms like Linux's Completely Fair Scheduler



Implement lightweight threads (green threads, goroutines) with kernel involvement



Avoid blocking I/O with asynchronous or non-blocking I/O operations



Process Management Summary

Process management is fundamental to operating system design, enabling efficient resource utilization and multitasking capabilities.



Processes

- ✓ Active entities with state and resources
- ✓ Dynamic execution instances



Process States

- ✓ New, Ready, Running, Waiting, Terminated
- ✓ Transitions driven by system events



Process Control Blocks

- ✓ Data structures for process management
- ✓ Store PID, memory layout, open files



Context Switching

- ✓ Save/restore process states
- ✓ Enables multitasking

💡 Key Insight: Understanding process management is **central to system performance** and reliability.